ATIVIDADE 02 – Protocolo IPv4

Nome: Matheus Milanezi

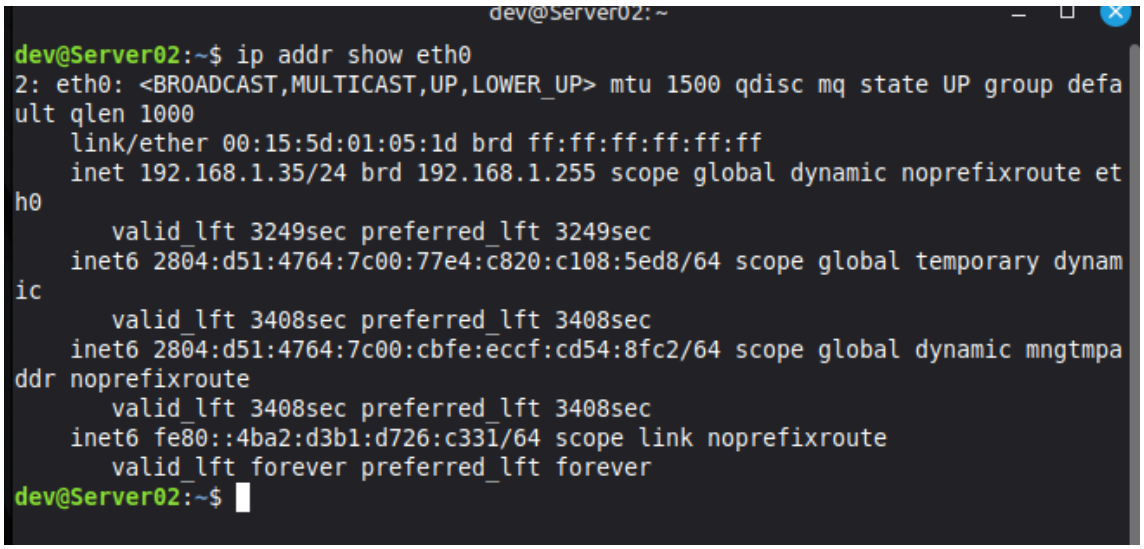
1. Ambiente de conectividade

O experimento foi realizado em uma máquina virtual criada no hipervisor Microsoft Hyper-V, com sistema operacional Linux Mint 22.3 (Cinnamon), com três processadores virtuais e memória RAM configurada de forma dinâmica entre 512MB a 10GB, kernel 6.17.0-35-generic. A conexão utilizada foi uma rede doméstica, através de interface Ethernet virtual (eth0), conectada ao roteador da residência (equipamento Nokia) via NAT/adaptador de rede da VM.

O host é um computador doméstico com sistema operacional Windows 10 PRO, processador Intel Core i5 – 9600k, 24GB de memória RAM.

Principais parâmetros identificados através do comando ip addr show eth0:

- Endereço IPv4: 192.168.1.35/24
- Máscara de rede: 255.255.255.0
- Gateway padrão: 192.168.1.254

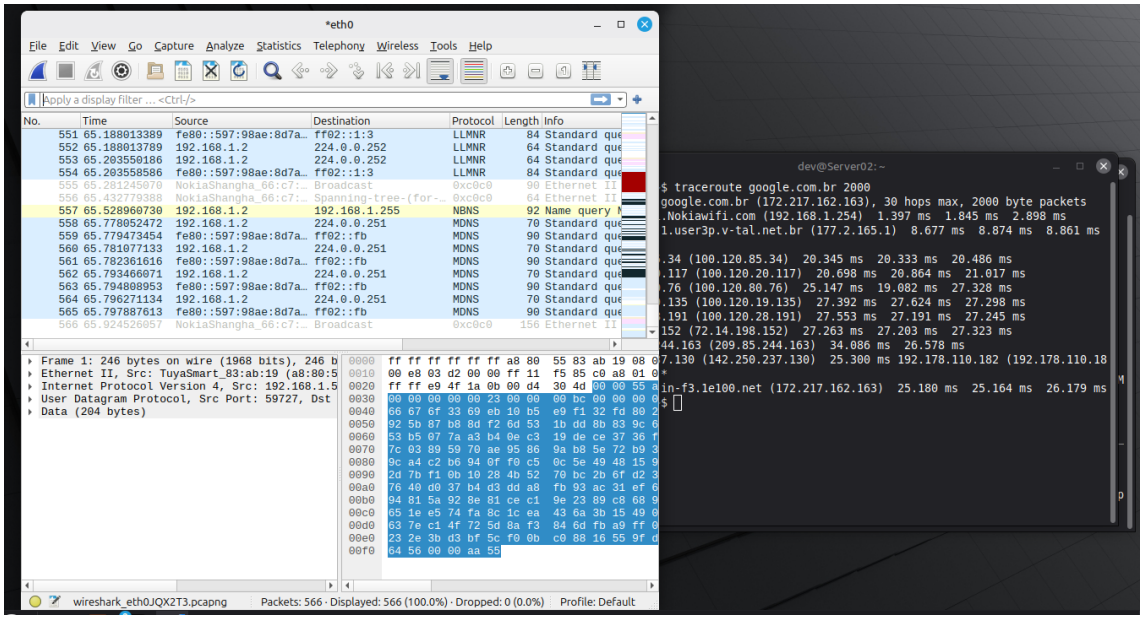


2. Execução do experimento e captura dos pacotes

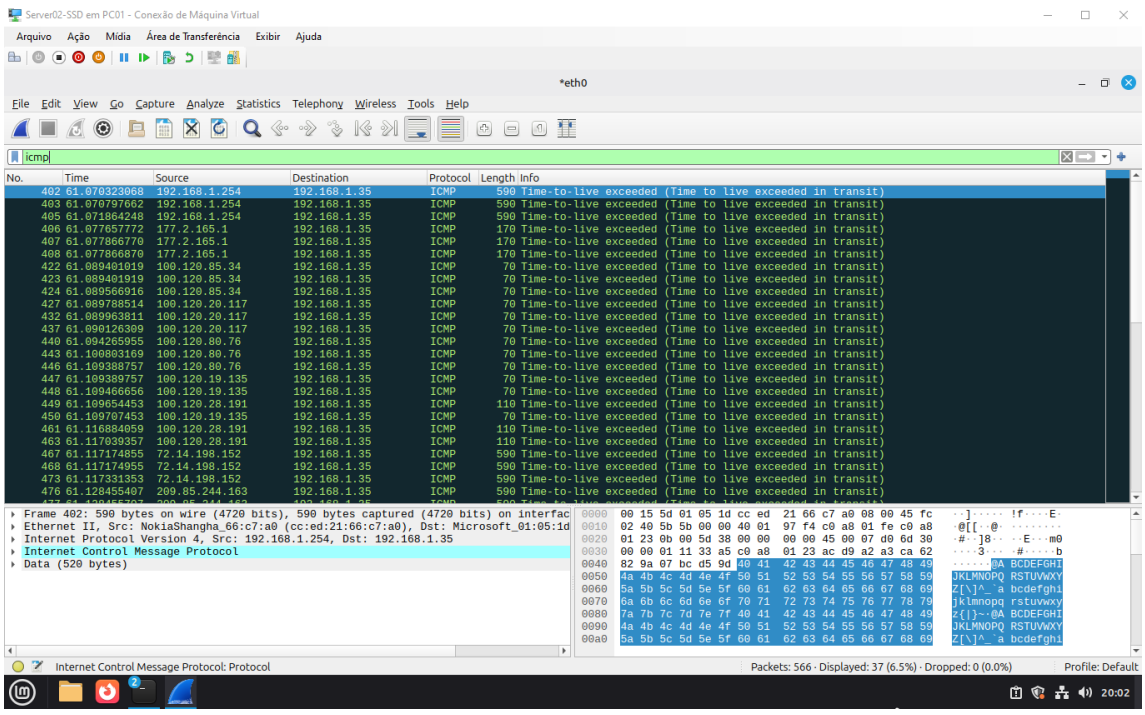
Foi iniciada uma captura de pacotes no Wireshark na interface eth0 e, em seguida, executado o comando abaixo em um segundo terminal, solicitando um datagrama de 2000 bytes:

$ traceroute google.com.br 2000

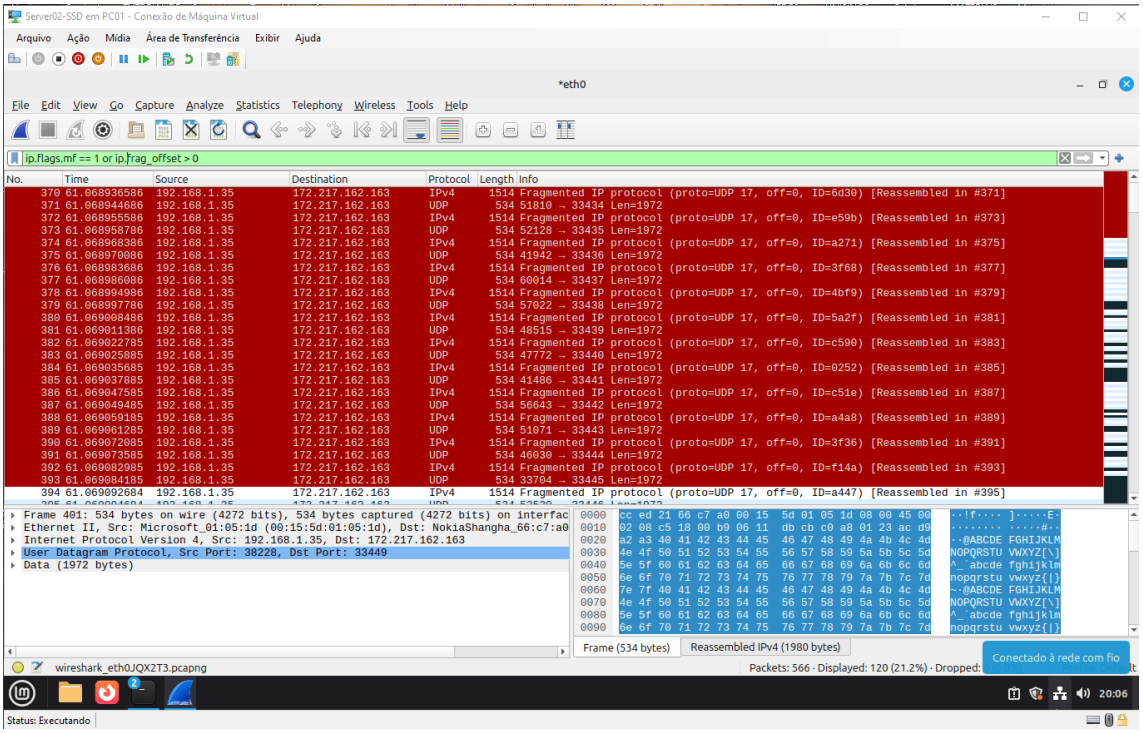
Segue abaixo prints do comando e da tela do Wireshark:



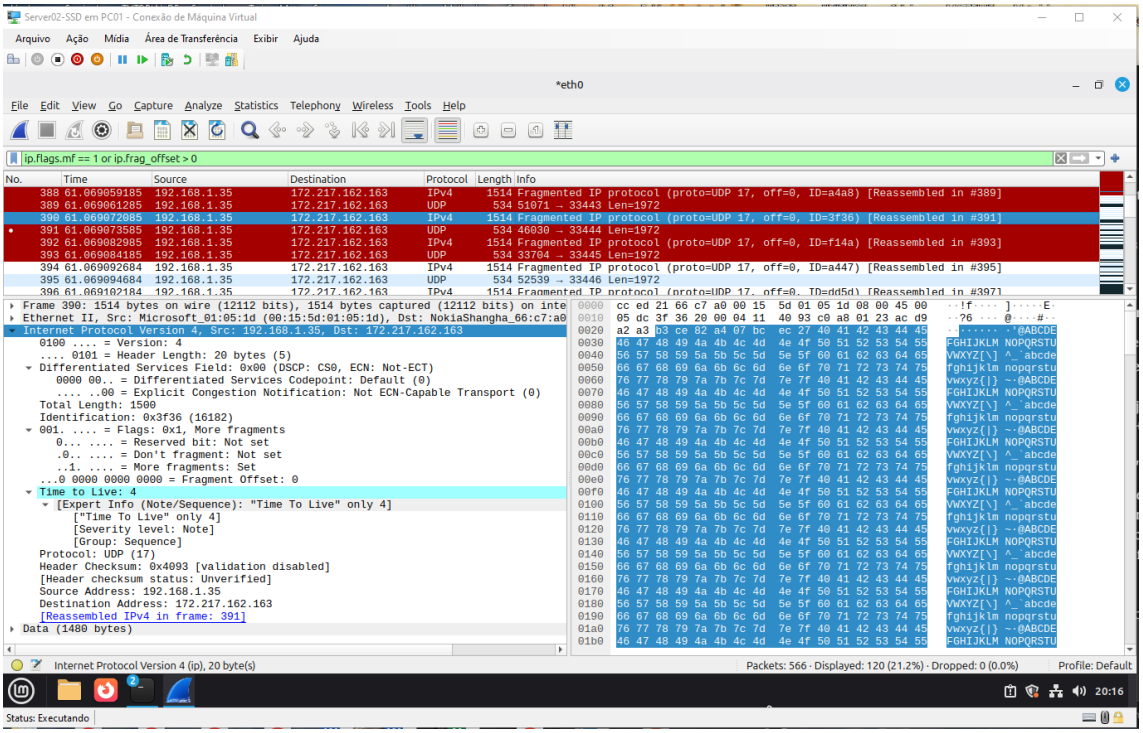
Após a captura, foi aplicado o filtro de exibição ICMP para isolar as respostas dos roteadores intermediários (mensagens Time Exceeded) e do destino final, conforme a figura abaixo:

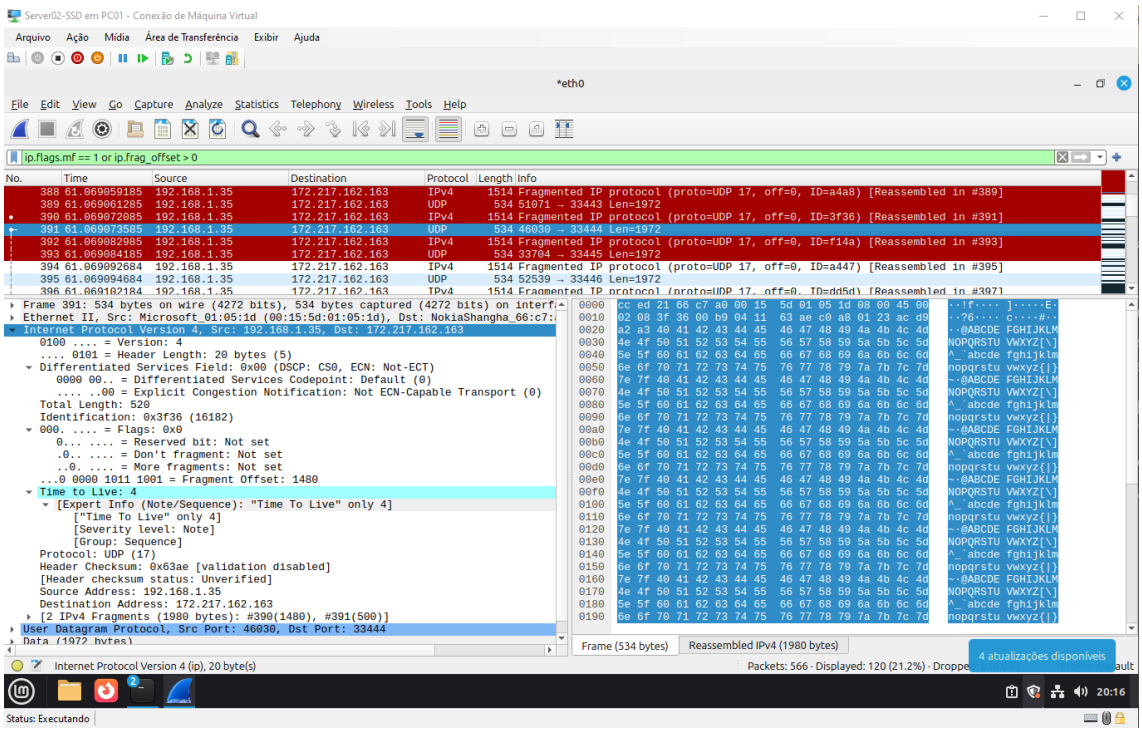


Em seguida, foi aplicado o filtro “ip.flags.mf == 1 or ip.frag_offset > 0” para isolar especificamente os fragmentos IPv4 gerados pelo datagrama de 2000 bytes, conforme figura abaixo:



Por fim, foram selecionados os dois fragmentos pertencentes ao mesmo datagrama (Identification 0x3f36, frames 390 e 391) para análise detalhada do cabeçalho IPv4, mostrados nas figuras abaixo:





3. Análise do datagrama IPv4 – Respostas:

1) O campo Protocol tem valor 17, que identifica o protocolo UDP. Isso é coerente com a implementação do traceroute no Linux, que por padrão usa pacotes UDP para sondar a rota (diferente do Windows, que usa ICMP por padrão).

2) O cabeçalho tem 20 bytes. No Wireshark isso aparece diretamente no campo “Header Length: 20 bytes (5)” o valor 5 representa a quantidade de palavras de 4 bytes (5 × 4 = 20). Esse é o tamanho mínimo do cabeçalho IPv4, sem campos de opções.

3) No fragmento 1 (frame 390): Total Length = 1500, menos 20 bytes de cabeçalho = 1480 bytes de payload. No fragmento 2 (frame 391): Total Length = 520, menos 20 bytes de cabeçalho = 500 bytes de payload. Somando os dois, obtém-se 1980 bytes de payload total, exatamente o valor confirmado pelo Wireshark onde “Reassembled IPv4 (1980 bytes)”. Somando-se o cabeçalho de 20 bytes, chega-se aos 2000 bytes solicitados no comando traceroute.

4) Sim, foi fragmentado. Os campos que comprovam isso são o Flags (no frame 390 o bit More Fragments está marcado como Set) e o Fragment Offset

(no frame 391 o offset é 1480, diferente de zero, indicando que aquele pacote é a continuação de um datagrama maior).

5) Foram gerados 2 fragmentos, detalhados na tabela abaixo:

Fragmento	Frame	Tamanho total / payload	Fragment Offset	Flag MF
1	390	1500 bytes / 1480 bytes	0	1 (Set)
2	391	520 bytes / 500 bytes	1480	0 (Not set)

6) O Identification é 0x3f36 e é o mesmo nos dois fragmentos. Esse campo é essencial porque é ele que permite ao host de destino identificar que os dois pedaços pertencem ao mesmo datagrama original, remontando-os na ordem correta com o auxílio do Fragment Offset. Sem esse campo, não seria possível diferenciar fragmentos de datagramas distintos trafegando simultaneamente na rede.

7) No par de fragmentos analisado (frames 390 e 391), o TTL enviado foi 4, correspondendo à sondagem do 4º salto da rota. Esse é o funcionamento característico do traceroute, ele envia pacotes com TTL crescente e cada roteador que recebe um pacote com TTL insuficiente descarta o pacote e devolve uma mensagem ICMP Time Exceeded para a origem. Isso foi observado na captura filtrada por icmp, com respostas sucessivas de 192.168.1.254, 177.2.165.1 e dos demais saltos, até o pacote alcançar o destino final (172.217.162.163), que respondeu com ICMP Destination Unreachable (Port Unreachable), sinal de chegada ao destino.

8) Sim, são totalmente coerentes. A interface eth0 tem MTU de 1500 bytes, confirmado no comando ip addr show. Como o datagrama solicitado tinha 2000 bytes, ultrapassando o limite da interface, o sistema operacional precisou fragmentá-lo antes do envio. O primeiro fragmento foi criado no tamanho máximo permitido pelo MTU (1500 bytes), e o restante (500 bytes) foi enviado no segundo fragmento.

4. Análise crítica do ambiente

O comportamento observado foi o esperado, e ver o processo acontecer na prática foi bastante interessante. O principal foi a fragmentação, sabia da existência do conceito de MTU e fragmentação, mas ele só faz sentido de forma completa quando se observa o próprio pacote sendo dividido em dois, com o mesmo Identification e Fragment Offset diferente, coincidindo exatamente com o tamanho do MTU da interface.

O ambiente de teste não pareceu interferir negativamente no resultado. A VM se comportou como qualquer máquina Linux comum na rede, onde recebeu IP via DHCP do roteador (192.168.1.35), utilizou o MTU padrão de Ethernet (1500) e não houve firewall ou NAT interno da própria VM atrapalhando a captura.

O que a captura de pacotes trouxe que a teoria sozinha não demonstra foi a visualização real dos bits dos campos, o flag More Fragments marcado (ou não marcado), o valor exato do Fragment Offset em bytes, o Identification se repetindo entre fragmentos, e as respostas ICMP surgindo na sequência correta conforme o TTL aumentava.

5. Uso de Inteligência Artificial

Foi utilizada a ferramenta Claude da Anthropic, como apoio ao longo de toda a atividade. A IA ajudou principalmente em duas etapas. Primeiro em explicar como configurar e usar o Wireshark explicando onde visualizar os dados, tamanho do datagrama e o que significava cada tela, já que era a primeira vez utilizando a ferramenta. Segundo indicar quais filtros de exibição utilizar (icmp e ip.flags.mf == 1 or ip.frag_offset > 0) para localizar os pacotes relevantes em meio as centenas de pacotes de tráfego irrelevante da rede. Todas as sugestões foram validadas com os dados reais da própria captura antes de serem aceitas, nenhum valor foi assumido apenas por confiança na IA.